

第12章 高级数据结构

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解红黑树五条性质与插入修正,知道 `std::map / set` 底层就是红黑树
- 掌握 Treap 与跳表的随机化平衡思想,能完整实现插入/删除/查找
- 掌握 k-d 树最近邻剪枝查询,理解配对堆摊还 $O(1)$ 合并
- 学会 map vs unordered_map 选型,并实现 10^4 级跳表字典与 `std::map` 实测对比

💡 从 C 到 C++:本章主题如何迁移

前 11 章最大的缺口是"平衡":BST 在有序输入下退化成链表,AVL 实现繁琐。本章五个结构给出三种哲学:红黑树用确定不变式强制平衡;Treap 与跳表把平衡交给随机性(期望 $O(\log n)$);配对堆靠摊还分析兜底。

在 C 里做"有序字典"很痛苦:手写 AVL/红黑树要几百行,一个修正分支写错就悄悄失衡;C++ 的答案是一行 `std::map` ——内部就是红黑树。

跳表与 Treap 的价值在"实现简单":代码量接近链表,却拿到期望 $O(\log n)$ ——用"概率上几乎不可能坏"换"写起来几乎不可能错"。

📦 前置知识自查

数学前置:概率期望直觉(几何分布)、对数运算、摊还分析概念(第11章)。

C 语言前置:`struct` 与指针、链表结点增删(第3章)、函数指针、`malloc / free` 对照 `new / delete`。

依赖的前面章节:第4章树(BST、旋转、AVL)、第5章散列、第6章堆、第11章摊还分析。

🚀 本章学习路线

1. **第一阶段(2 小时):**读 §12.1–§12.4,背下修正口诀,手写 Treap、跳表与 k-d 树
2. **第二阶段(1 小时):**读 §12.5–§12.6,理解配对堆与选型决策表
3. **第三阶段(实验,1 小时):**完成章末实验:跳表字典的实现与对比
4. **验收标准:**能随口说出"红黑树高度 $O(\log n)$ " "跳表期望 $O(\log n)$ "

本章知识导图

- §12.1 红黑树:五条性质与插入修正
- §12.2 Treap:随机优先级的树堆
- §12.3 跳表:多层链表的概率平衡
- §12.4 k-d 树:多维空间切分与最近邻
- §12.5 配对堆:摊还 $O(1)$ 合并的实用堆
- §12.6 标准库选型指南
- §12.7 全书总结:十二章知识地图

§12.1 红黑树:五条性质与插入修正

12.1.1 五条性质与"黑高"

红黑树是一棵二叉查找树,每个结点带红/黑一色,满足五条性质:① 结点非红即黑;② 根是黑色;③ 叶结点(NIL)是黑色;④ 红结点的子结点都是黑色——"红结点不能有红孩子";⑤ 从任一结点到其所有后代叶的路径上,黑色结点数相同,这个数叫**黑高**。

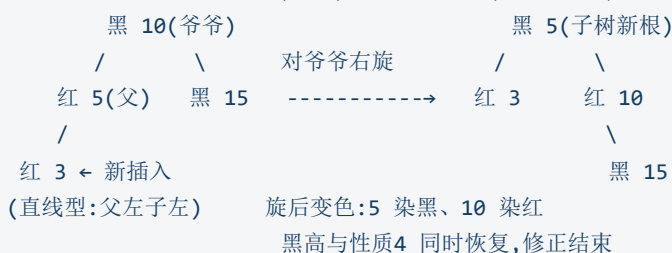
关键保证:设黑高 h_b ,最短路径(全黑)长 h_b ,性质4 又禁红红相邻,最长路径至多 $2h_b$ ——最长 $\leq$ 最短的 2 倍,树高 $O(\log n)$,三操作最坏 $O(\log n)$ 。这就是"弱平衡":不追求 AVL 的高度差 ≤ 1 。

12.1.2 插入修正:变色与旋转

插入先染**红**——红结点不影响性质5(黑高不变),只可能破坏性质4(红父红子)。修正只看叔叔颜色:叔叔红 $\rightarrow$ 变色;叔叔黑 $\rightarrow$ 旋转。

示例 12-1 插入修正:叔叔为黑 $\rightarrow$ 旋转 + 变色

红黑树插入修正之二:叔叔为黑(或空),旋转 + 变色(LL 型为例)



叔叔为红时,父、叔变黑,爷爷变红,冲突上移一层继续检查;爷爷若是根最后染黑。叔叔为黑时按 AVL 的四种形态旋转:LL/RR 直线一次旋转,LR/RL 之字形两次旋转(先转父再转爷爷);旋转后被提上来的结点染黑、爷爷染红。

★ 重点:插入易、删除难

插入只处理"红父红子"一处冲突,沿路径上移 $O(\log n)$ 层;删除要把黑高"挖掉"一层,处理"双黑"问题——红黑树"实现极复杂、接口极简单",删除从不手写,因此适合内建在标准库。

12.1.3 与 2-3-4 树的关系

一句话:**红黑树是 2-3-4 树的二叉表示**——把 2-3-4 树一个结点内的 2~3 个键"压扁"成红黑交替的二叉链。

12.1.4 C++ 视角:STL 的 map/set 就是红黑树

`std::map` / `std::set` 底层就是红黑树,于是 map 有三个赠品:① 中序遍历即按键升序;② 插入/删除其他元素不使既有迭代器失效;③ `lower_bound` / `upper_bound` $O(\log n)$ 定位区间。

C 的手写有序查找表	C++ 的标准库
手写 AVL/红黑树:数百行,修正分支众多	<code>std::map</code> / <code>std::set</code> :一行声明即用
有序遍历要自己写中序递归	范围 <code>for</code> 直接按键序输出
范围查询手写"第一个 $\geq x$ 的结点"	<code>lower_bound</code> / <code>upper_bound</code> 现成
忘记 <code>free</code> 整棵树即泄漏	析构自动释放全部结点

⚠ 误区 1:性质4 的方向与口诀记反

性质4 是"红结点的孩子必须黑",反过来"黑结点的孩子必须红"是错的——黑结点孩子颜色任意。高频错误还有口诀记反:叔叔红时去旋转,树永远修不好。

§12.2 Treap:随机优先级的树堆

12.2.1 思想:BST + 堆序

BST 退化的根源是"输入顺序决定树形"。Treap 换一条路:**Treap**(tree+heap)让每个结点携带一个**随机优先级**,并额外要求优先级满足**堆序**(小顶堆:父 $\leq$ 子)。键决定"在哪里",优先级决定"谁当根"——优先级随机,树形就随机,期望高度 $O(\log n)$ 。把平衡从"旋转纠错"换成"随机决定",实现从几百行降到几十行。

12.2.2 插入与旋转

插入:先按 BST 规则插到叶位置,再沿路径向上旋转,直到父的优先级 $\leq$ 子——与二叉堆"上滤"异曲同工。删除同理:把目标旋转着推下去,变叶子再摘除。

示例 12-5 Treap 完整实现(第一段:结构、旋转、插入)

```

// Treap:键满足 BST 序,随机优先级满足小顶堆序
#include <iostream>
#include <random>
using namespace std;

struct Node {
    int key, pri;
    Node* left = nullptr;
    Node* right = nullptr;
    Node(int k, int p) : key(k), pri(p) {}
};

Node* rotateRight(Node* t) {    // 右旋:提升左孩子
    Node* x = t->left;
    t->left = x->right;
    x->right = t;
    return x;
}

Node* rotateLeft(Node* t) {    // 左旋:提升右孩子
    Node* x = t->right;
    t->right = x->left;
    x->left = t;
    return x;
}

Node* insert(Node* t, int key, int pri) {
    if (!t) return new Node(key, pri);
    if (key < t->key) {
        t->left = insert(t->left, key, pri);
        if (t->left->pri < t->pri) t = rotateRight(t); // 堆序上滤
    } else if (key > t->key) {
        t->right = insert(t->right, key, pri);
        if (t->right->pri < t->pri) t = rotateLeft(t);
    }
    return t;
}

```

示例 12-6 Treap 完整实现(第二段:删除与查找)

```

Node* erase(Node* t, int key) {
    if (!t) return nullptr;
    if (key < t->key) t->left = erase(t->left, key);
    else if (key > t->key) t->right = erase(t->right, key);
    else if (!t->left) { Node* r = t->right; delete t; return r; }
    else if (!t->right) { Node* l = t->left; delete t; return l; }
    else if (t->left->pri < t->right->pri) {
        Node* x = rotateRight(t);
        x->right = erase(t, key);
        return x;
    } else {
        Node* x = rotateLeft(t);
        x->left = erase(t, key);
        return x;
    }
    return t;
}

bool find(Node* t, int key) {
    while (t && t->key != key)
        t = key < t->key ? t->left : t->right;
    return t != nullptr;
}

```

12.2.3 期望 $O(\log n)$ 的直觉与缺陷

直觉:根是整棵树优先级最小的结点,优先级随机,所以"根是第 k 小的键"的概率均匀为 $1/n$,左右子树递归同构——与快排"随机枢轴"的分析同构:Treap 就是快排的树形版本。最坏情形(优先级恰好按键单调)概率 $1/n!$,现实不会出现。优先级相等时须约定比较方向,否则可能破坏 BST 序。

⚠ 误区 2:堆序方向与旋转方向配错

本例用小顶堆(父 $\leq$ 子),口诀"左小右旋、右小左旋";若定义成大顶堆,所有比较都要取反。实现后用有序序列插入再中序遍历:输出必须升序,且每个结点优先级 $\leq$ 子结点。

§12.3 跳表:多层链表的概率平衡

12.3.1 多层链表与几何分布

有序链表查找 $O(n)$,因为每步只能走一个结点。跳表(skip list)的思路:多建几层链表,上层是下层的"快速通道"。每个新结点先进入最底层,再以概率 $1/2$ 晋升,再以概率 $1/2$ 继续.....层数 k 满足几何分布: $P(\text{层} \geq k) = 2^{-(k-1)}$,每层结点数期望 $n/2^{k-1}$ 。查找从最高层出发,每层右移到"下一个更大"结点前再下降,共 $O(\log n)$ 层。

12.3.2 三操作:查找、插入、删除

三操作共享同一"下探"骨架:从最高层向右试探,把每层"最后一个 $< \text{key}$ 的结点"记进 `update` 数组。**查找**:下探后看 `update[0]->next[0]` 是否为 `key`。**插入**:随机层数 `lvl`,若高于当前最高层先补 `update` 头指针,再逐层链入。**删除**:下探后若第 0 层后继即目标,逐层跳过并 `delete`,最后回收空高层。

示例 12-7 跳表完整实现(第一段:结构、查找、插入)

```

#include <iostream>
#include <random>
#include <vector>
using namespace std;

struct Node {
    int key;
    vector<Node*> next;          // next[i]:第 i 层的后继
    Node(int k, int lvl) : key(k), next(lvl + 1, nullptr) {}
};

class SkipList {
    Node* head_;                // 头结点:全部层的起点
    int level_ = 0;
    mt19937 rng_{12345};        // 固定种子
public:
    SkipList(int maxLvl = 16) : head_(new Node(0, maxLvl)) {}
    ~SkipList() {
        Node* p = head_->next[0];
        while (p) { Node* t = p; p = p->next[0]; delete t; }
        delete head_;
    }
    int randomLevel() {          // 几何分布:每层以 1/2 概率继续
        int lvl = 1;
        while ((rng_() & 1) && lvl < int(head_->next.size()) - 1) ++lvl;
        return lvl;
    }
    bool find(int key) const {
        Node* cur = head_;
        for (int i = level_; i >= 0; --i)
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
        Node* x = cur->next[0];
        return x != nullptr && x->key == key;
    }
    void insert(int key) {
        if (find(key)) return;
        vector<Node*> update(head_->next.size());
        Node* cur = head_;
        for (int i = level_; i >= 0; --i) {
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
            update[i] = cur;
        }
        int lvl = randomLevel();
        if (lvl > level_) {
            for (int i = level_ + 1; i <= lvl; ++i) update[i] = head_;
            level_ = lvl;
        }
        Node* x = new Node(key, lvl);
        for (int i = 0; i <= lvl; ++i) {
            x->next[i] = update[i]->next[i];
            update[i]->next[i] = x;
        }
    }
};

```

```

    }
}

```

示例 12-8 跳表完整实现(第二段:删除与按序遍历)

```

bool erase(int key) {
    vector<Node*> update(head_>next.size());
    Node* cur = head_;
    for (int i = level_; i >= 0; --i) {
        while (cur->next[i] && cur->next[i]->key < key)
            cur = cur->next[i];
        update[i] = cur;
    }
    Node* x = cur->next[0];
    if (!x || x->key != key) return false;
    for (int i = 0; i <= level_; ++i) {
        if (update[i]->next[i] != x) break;
        update[i]->next[i] = x->next[i];
    }
    delete x;
    while (level_ > 0 && head_>next[level_] == nullptr)
        --level_;
    return true;
}

vector<int> keys() const {
    vector<int> v;
    for (Node* p = head_>next[0]; p; p = p->next[0]) v.push_back(p->key);
    return v;
}

size_t size() const { return keys().size(); }
};

```

12.3.3 与平衡树对比:期望 vs 确定

对比项	跳表	红黑树 / AVL
平衡机制 / 实现难度	随机层数, ~ 链表级, 几十行	旋转 + 不变式, 高, 数百行
高度	期望 $O(\log n)$, 最坏 $O(n)$	最坏保证 $O(\log n)$
每结点内存	平均约 2 个指针	2 个指针 + 颜色位
工程使用	Redis 有序集合 zset、LevelDB	std::map/set、Java TreeMap

C 的手写多级链表	C++ 的跳表类
每层一个 Node* next[k] 数组要手写分配	vector<Node*> next 自动管理层指针
释放多层结点容易重复 free 或泄漏	析构沿第 0 层逐个 delete, RAIL 兜底
随机层数用 rand(), 可移植性差	mt19937 质量与可复现性都有保证

⚠ 误区 3:随机层数用错了分布

层数必须**几何分布**(每层 1/2 概率继续)。三个常见错误:① 固定层数让高层退化成线性扫描 $O(n)$;② 删除后不回收空高层;③ 用 `time()` 做种子,同一秒插入的结点层数高度相关。调试:逐层打印链表,验证"第 0 层完整、高层稀疏"。

§12.4 k-d 树:多维空间切分与最近邻

12.4.1 交替维度切分建树

k-d 树递归地选一个维度取**中位数**作切分点,左子树为小于切分点的点集,右子树反之;下一层换另一个维度。对二维点,第 0 层按 x 切、第 1 层按 y 切、第 2 层再按 x 维度交替,保证每个区域在两个方向都被"削"过,树高 $O(\log n)$ 。

示例 12-9 k-d 树:交替维度建树(二维)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Point { double x, y; };

struct KdNode {
    Point p;
    int dim;           // 切分维度:0 按 x,1 按 y
    KdNode* left = nullptr;
    KdNode* right = nullptr;
    KdNode(const Point& pt, int d) : p(pt), dim(d) {}
};

// nth_element:O(n) 选出中位数并原地分区
KdNode* build(vector<Point>& pts, int lo, int hi, int depth) {
    if (lo > hi) return nullptr;
    int dim = depth % 2;
    int mid = (lo + hi) / 2;
    nth_element(pts.begin() + lo, pts.begin() + mid, pts.begin() + hi + 1,
        [dim](const Point& a, const Point& b) {
            return dim == 0 ? a.x < b.x : a.y < b.y;
        });
    KdNode* node = new KdNode(pts[mid], dim);
    node->left = build(pts, lo, mid - 1, depth + 1);
    node->right = build(pts, mid + 1, hi, depth + 1);
    return node;
}
```

12.4.2 最近邻查询与剪枝

给定查询点 q ,找点集中距离最近者。先沿树下探到 q 所在区域,得到"当前最佳距离" d_{best} ;回溯时——若 q 到分割线的距离 $\geq d_{\text{best}}$,另一侧任何点都不可能更近,整棵子树剪掉。

示例 12-10 k-d 树:最近邻查询(带剪枝)


```
double dist2(const Point& a, const Point& b) {
    double dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy;
}

void nearest(KdNode* node, const Point& q, Point& best, double& bestD) {
    if (!node) return;
    double d = dist2(node->p, q);
    if (d < bestD) { bestD = d; best = node->p; }

    int dim = node->dim;
    double cut = dim == 0 ? node->p.x : node->p.y;
    double qc = dim == 0 ? q.x : q.y;
    KdNode* near = (qc < cut) ? node->left : node->right;
    KdNode* far = (qc < cut) ? node->right : node->left;
    nearest(near, q, best, bestD);

    double dCut = (qc - cut) * (qc - cut);
    if (dCut < bestD)
        nearest(far, q, best, bestD);
}
```

记号说明:距离平方省去开根号

最近邻只需比较大小, `dist2` 返回距离平方即可,省掉每步一次 `sqrt`;剪枝条件用平方比较数学等价。

12.4.3 应用与退化警告

k-d 树是"多维点集查询"的默认答案:游戏引擎的"最近敌人"查询替代逐对检测的 $O(n^2)$,光线追踪与 k-NN 分类也建立在"空间切分 + 剪枝"上。数据挤在一条直线上时剪枝失效,查询最坏 $O(n)$;生产级替代是 R 树或网格索引。

误区 4:剪枝条件漏写或写反

最近邻两种经典错误:① 漏掉 `if (dCut < bestD)` 直接递归两侧——正确但退化成全树搜索 $O(n)$;② 比较方向反了——可能跳过真正更近的点。调试:拿 5 点小数据集逐结点打印"近侧/远侧/是否剪枝"核对。

§12.5 配对堆:摊还 $O(1)$ 合并的实用堆

12.5.1 配对堆是什么

第6章的二叉堆支持 `insert` 与 `deleteMin`,但**合并两个堆**要 $O(n)$ 。左式堆 `merge` 是 $O(\log n)$ 但实现复杂;**配对堆** (pairing heap)走实用路线: `merge` 只比两个根的键,把小根挂到大根的孩子链表里,摊还 $O(1)$; `deleteMin` 把根的孩子两两配对合并,摊还 $O(\log n)$ 。

示例 12-11 配对堆:`merge`、`insert`、`deleteMin`

```
// 配对堆:孩子用"左孩子 + 右兄弟"链表表示,根最小
struct PHNode {
    int val;
    PHNode* child = nullptr;    // 第一个孩子,next 为兄弟链
    PHNode* next = nullptr;
    PHNode(int v) : val(v) {}
};

PHNode* merge(PHNode* a, PHNode* b) {    // 摊还  $O(1)$ 
    if (!a) return b;
    if (!b) return a;
    if (a->val > b->val) swap(a, b);    // 键小的做根
    b->next = a->child;
    a->child = b;
    return a;
}

PHNode* mergePairs(PHNode* first) {    // 两趟合并
    if (!first || !first->next) return first;
    PHNode* a = first;
    PHNode* b = first->next;
    PHNode* rest = b->next;
    return merge(merge(a, b), mergePairs(rest));
}

PHNode* deleteMin(PHNode* root) {
    PHNode* children = root->child;
    delete root;
    return mergePairs(children);
}

// insert = merge(现堆, 单结点堆),摊还  $O(1)$ 
```

12.5.2 三堆选择的一句话

只做优先队列(不合并):二叉堆;需要合并且代码量敏感:配对堆;需要最坏保证的合并:左式堆。代价是摊还而非最坏:单次 `deleteMin` 可能偶发 $O(n)$,长时间平均 $O(\log n)$ 。

§12.6 标准库选型指南

12.6.1 map/set vs unordered_map/set:决策表

`map / set` 底层红黑树, `unordered_map / unordered_set` 底层散列表(第5章)。选哪个:

需求	map / set(红黑树)	unordered_map / set(散列)
按键升序遍历	支持(中序遍历)	不支持(顺序随机)
范围查询 [lo, hi]	支持(lower_bound/upper_bound)	不支持
平均查找速度	$O(\log n)$	$O(1)$,常数更小
最坏查找	$O(\log n)$ 有保证	$O(n)$ (散列冲突)
迭代器失效	插入不失效,删除只失效被删者	再散列(rehash)时全部失效

12.6.2 自定义比较器与自定义哈希

键是自定义结构时,map 需要"严格弱序"(等价的键互比为假),unordered_map 需要哈希与判等:

示例 12-12 自定义比较器与自定义哈希

```
#include <map>
#include <unordered_map>
#include <string>
using namespace std;

struct Person { string name; int age; };

struct ByAge {
    // 自定义比较器:按年龄排序
    bool operator()(const Person& a, const Person& b) const {
        return a.age < b.age; // 严格弱序:age 相同视为等价
    }
};

map<Person, int, ByAge> ageRank;

struct PersonHash {
    // 自定义哈希:让 Person 可散列
    size_t operator()(const Person& p) const {
        return hash<string>()(p.name) ^ (hash<int>()(p.age) << 1);
    }
};

unordered_map<Person, int, PersonHash> phone;
```

C 的做法	C++ 的做法
qsort(a, n, sz, cmp) 传函数指针,比较逻辑与容器分离	map<K, V, ByAge> 传类型(函数对象/lambda),随容器封装
手写散列表,冲突处理自己维护	unordered_map<K, V, Hash> 只需提供哈希类型
比较器写错静默产生错误排序	严格弱序违规是未定义行为,用断言检查等价性

12.6.3 手写 vs 标准库:有序字典并排

把 §12.3 的跳表包一层,接口与 std::map 对齐——第3章 ADT 的威力:

示例 12-13 手写跳表字典 vs std::map,接口并排

```
// ---- 手写版:Skiplist 包成有序字典 ----
class SkipDict {
    SkipList sl_;
public:
    void insert(int k) { sl_.insert(k); }
    bool find(int k)   { return sl_.find(k); }
    bool erase(int k)  { return sl_.erase(k); }
    vector<int> keys() { return sl_.keys(); }
};

SkipDict dict;
dict.insert(5); dict.insert(3);
vector<int> ord = dict.keys();           // [3, 5]

// ---- 标准库版:同一套接口,红黑树 ----
map<int, string> m;
m[5] = "five"; m[3] = "three"; m[8] = "eight";
for (auto& [k, v] : m) { /* 按序遍历 */ }
auto it = m.lower_bound(4);
```

12.6.4 工程选型建议

- 一次性有序输出:vector + `sort`
- 持续插入 + 需要有序:map(手写红黑树永远不是选项)
- 只查不遍历:unordered_map;冲突多时换更均匀的哈希
- Python 同学没有 map:list+bisect、dict+sorted、sortedcontainers(见 Python 版 §12.6)

§12.7 全书总结:十二章知识地图

12.7.1 十二章知识地图

- 第1章 绪论:递归与数学 → 第2章 算法分析: $O/\Omega/\Theta$
- 第3章 表栈队列:数组与链表 → 第4章 树:BST、AVL、遍历
- 第5章 散列:冲突处理 → 第6章 堆:二叉堆、上滤下滤
- 第7章 排序:八大排序 → 第8章 不相交集:union/find
- 第9章 图:最短路与 MST → 第10章 算法设计:贪心、分治、DP
- 第11章 摊还分析:三方法 → 第12章 高级结构:平衡与期望

12.7.2 "遇到问题选什么结构"终极决策表

需求场景	推荐结构	理由
只做等值查找/去重	散列(unordered_map / dict)	平均 $O(1)$, 常数最小
需要按键有序遍历	红黑树(map)或跳表	中序/底层链即有序序列
范围查询 [lo, hi]	map 的 lower_bound / 有序数组二分	$O(\log n)$ 定位 + $O(k)$ 输出
优先队列(含合并)	二叉堆 / 配对堆	$O(\log n)$ / 合并摊还 $O(1)$
多维点集最近邻	k-d 树	平均 $O(\log n)$ 剪枝查询

12.7.3 下一步学习方向

学完十二章,你已具备"手写任何基础结构 + 正确选型"的能力。三个方向任选:① **算法竞赛**——跳表与 Treap 常作为平衡树的快速替代;② **系统设计**——缓存用散列、排行榜用跳表、范围索引用 B+ 树;③ **数据库索引**——B+ 树是红黑树的"磁盘亲戚"。最后记住:数据结构没有银弹,选型的本质是拿时间复杂度、实现复杂度与最坏保证做三角权衡。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找 / 插入 / 删除	红黑树(map/set 底层)	$O(\log n)$	$O(\log n)$
有序遍历全部元素	红黑树中序遍历	$O(n)$	$O(n)$
前驱 / 后继	红黑树迭代器 ++/--	$O(\log n)$	$O(\log n)$
查找 / 插入 / 删除	Treap	期望 $O(\log n)$	$O(n)$ (优先级恰好有序)
查找 / 插入 / 删除	跳表	期望 $O(\log n)$	$O(n)$ (全部同层)
建树	k-d 树(nth_element 切分)	$O(n \log n)$	$O(n \log n)$
最近邻查询	k-d 树	平均 $O(\log n)$	$O(n)$ (数据退化)
合并两个堆	配对堆	摊还 $O(1)$	摊还 $O(1)$
插入(单结点)	配对堆	摊还 $O(1)$	摊还 $O(1)$
deleteMin	配对堆	摊还 $O(\log n)$	摊还 $O(\log n)$
查找	map / set	$O(\log n)$	$O(\log n)$
查找	unordered_map / unordered_set	平均 $O(1)$	$O(n)$ (散列冲突)
范围查询 [lo, hi]	map 的 lower_bound/upper_bound	$O(\log n) + O(k)$	同左(k 为输出个数)
二分查找	有序数组 + std::binary_search	$O(\log n)$	$O(\log n)$
插入到有序数组	vector + insert	$O(n)$	$O(n)$

最小必会清单

- 能默写红黑树五条性质,并解释"性质4+性质5 $\rightarrow$ 高度 $O(\log n)$ "
- 能画出插入修正两个分支(叔叔红变色、叔叔黑旋转)与四种旋转形态
- 能说出"红黑树是 2-3-4 树的二叉表示"
- 能独立实现 Treap 与跳表的插入/删除/查找,说出期望为何 $O(\log n)$
- 能说出跳表与红黑树的取舍(期望 vs 确定、实现难度、内存、并发)
- 能独立实现 k-d 树交替维度建树与最近邻剪枝查询
- 能说出配对堆合并摊还 $O(1)$ 、deleteMin 摊还 $O(\log n)$,一句话对比三堆
- 能背出 map vs unordered_map 决策表,写出自定义比较器(严格弱序)与自定义哈希

自测题

Q 1. 为什么性质4 与性质5 一起能保证红黑树高度 $O(\log n)$?

✓ 参考答案

性质5 说所有路径黑高相同,设黑高 h_b ;性质4 禁红红相邻,最长路径至多 $2h_b$,最短路径长 h_b ——最长 $\leq 2 \times$ 最短。而黑高 h_b 的树至少 $2^{h_b}-1$ 个内部结点,故树高 $\leq 2 \log_2(n+1) = O(\log n)$ 。

Q 2. Treap 的期望高度为什么是 $O(\log n)$?最坏情形是什么?

✓ 参考答案

优先级随机,根是"优先级最小结点"的概率均匀为 $1/n$,左右子树递归同构——与快排随机枢轴同构。最坏是优先级恰好按键单调,高度 $O(n)$,概率 $1/n!$ 。

Q 3. 跳表查找从第几层开始?层数为什么用几何分布而不是固定层数?

✓ 参考答案

从当前最高层开始逐层下降。几何分布(每层 $1/2$ 概率晋升)使每层期望结点数 $n/2^k$,查找每层只走常数步,期望 $O(\log n)$ 。固定层数则退化成线性扫描。

Q 4. k-d 树最近邻查询的剪枝条件是什么?漏写会怎样?

✓ 参考答案

回溯时,若查询点到分割线的距离平方 $\geq$ 当前最佳距离平方,另一侧不可能更近,整棵子树剪掉;只有严格小于才进入。漏写则退化成全树搜索 $O(n)$;方向写反则可能漏掉更近的点。

Q 5. 向 map 与 unordered_map 各插入一个元素,既有迭代器分别会怎样?工程含义?

✓ 参考答案

map(红黑树)插入只改指针不搬内存,除被删除者外所有迭代器保持有效;unordered_map 插入可能触发再散列,全部迭代器失效。"遍历中插入"或"缓存迭代器"必须优先 map——§3.3 迭代器失效思想再现。

章末实验题:跳表字典的实现与对比

📄 实验 12:跳表字典的实现与对比

实验目标:实现完整跳表,封装成有序字典,与 `std::map` 对照验证正确性并实测 10^4 级随机键耗时。

实验要求:

- 任务 1:实现跳表的查找/插入/删除与随机层数(知识点:§12.3 三操作与几何分布)
- 任务 2:封装成有序字典接口 insert/find/erase/keys(知识点:§12.3 底层链有序 + 第3章 ADT)
- 任务 3:用 `std::map` 做同一组随机操作,验证输出一致(知识点:§12.6 标准库对照)
- 任务 4:对 10^4 级随机键分别计时插入与查找(知识点:§12.3 期望复杂度 + 第2章计时)
- 任务 5:删除全部元素,验证跳表为空(知识点:§12.3 删除与空层回收)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab12.cpp -o lab12` ;运行 `lab12` 。

第一步 示例 12-7/12-8 实现 SkipList,升/降/乱序三组输入冒烟测试,遍历必须升序

第二步 任务 3 用同一种子生成随机操作序列,插入前先 find 去重;任务 4 洗牌生成 10^4 个不重复键,结果累加输出防死代码消除

✔ 参考答案(完整可运行程序,约 140 行,分四段)


```

// lab12.cpp — 跳表字典的实现与对比(第一段:任务1 跳表核心)
#include <iostream>
#include <vector>
#include <random>
#include <map>
#include <algorithm>
#include <chrono>
using namespace std;
using Clock = chrono::steady_clock;

struct Node {
    int key;
    vector<Node*> next;
    Node(int k, int lvl) : key(k), next(lvl + 1, nullptr) {}
};

class SkipList {
    // 任务 1:三操作 + 随机层数
    Node* head_;
    int level_ = 0;
    mt19937 rng_{12345}; // 固定种子:可复现
public:
    SkipList(int maxLvl = 16) : head_(new Node(0, maxLvl)) {}
    ~SkipList() {
        Node* p = head_>next[0];
        while (p) { Node* t = p; p = p->next[0]; delete t; }
        delete head_;
    }
    int randomLevel() { // 几何分布:每层 1/2 概率晋升
        int lvl = 1;
        while ((rng_() & 1) && lvl < int(head_>next.size()) - 1) ++lvl;
        return lvl;
    }
    bool find(int key) const {
        Node* cur = head_;
        for (int i = level_; i >= 0; --i)
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
        Node* x = cur->next[0];
        return x != nullptr && x->key == key;
    }
    void insert(int key) {
        if (find(key)) return; // 字典语义:键唯一
        vector<Node*> update(head_>next.size());
        Node* cur = head_;
        for (int i = level_; i >= 0; --i) {
            while (cur->next[i] && cur->next[i]->key < key)
                cur = cur->next[i];
            update[i] = cur;
        }
        int lvl = randomLevel();
        if (lvl > level_) {
            for (int i = level_ + 1; i <= lvl; ++i) update[i] = head_;
            level_ = lvl;
        }
        Node* x = new Node(key, lvl);
    }
};

```

```

        for (int i = 0; i <= lvl; ++i) {
            x->next[i] = update[i]->next[i];
            update[i]->next[i] = x;
        }
    }
}

```

lab12.cpp(第二段:删除、遍历 + 任务2 字典封装)

```

bool erase(int key) {
    vector<Node*> update(head_->next.size());
    Node* cur = head_;
    for (int i = level_; i >= 0; --i) {
        while (cur->next[i] && cur->next[i]->key < key)
            cur = cur->next[i];
        update[i] = cur;
    }
    Node* x = cur->next[0];
    if (!x || x->key != key) return false;
    for (int i = 0; i <= level_; ++i) {
        if (update[i]->next[i] != x) break;
        update[i]->next[i] = x->next[i];
    }
    delete x;
    while (level_ > 0 && head_->next[level_] == nullptr) --level_;
    return true;
}

vector<int> keys() const {
    vector<int> v;
    for (Node* p = head_->next[0]; p; p = p->next[0]) v.push_back(p->key);
    return v;
}

size_t size() const { return keys().size(); }
};

class SkipDict {
    // 任务 2:有序字典接口
    SkipList sl_;
public:
    void insert(int k) { sl_.insert(k); }
    bool find(int k) { return sl_.find(k); }
    bool erase(int k) { return sl_.erase(k); }
    vector<int> keys() { return sl_.keys(); }
    size_t size() const { return sl_.size(); }
};

```

lab12.cpp(第三段:任务3 一致性对照)

```

int main() {
    // 任务 3:与 std::map 对照,输出必须一致
    cout << "== 任务3:跳表字典 vs std::map 一致性 ==" << endl;
    SkipDict dict;
    map<int, int> m;
    mt19937 rng(12345);
    uniform_int_distribution<int> dist(0, 99999);
    for (int i = 0; i < 20000; ++i) {
        int k = dist(rng);
        int op = i % 4;           // 0/1 插,2 删,3 查
        if (op <= 1) {
            if (!m.count(k)) dict.insert(k);
            m[k] = k;
        } else if (op == 2) {
            dict.erase(k);
            m.erase(k);
        } else {
            bool a = dict.find(k), b = m.count(k) > 0;
            if (a != b) { cout << "不一致! key=" << k << endl; return 1; }
        }
    }
    vector<int> dk = dict.keys(), mk;
    for (auto& p : m) mk.push_back(p.first);
    cout << "20000 次随机操作后:跳表 size=" << dict.size()
        << ",map size=" << m.size() << endl;
    cout << "跳表前 10 键: ";
    for (int i = 0; i < 10; ++i) cout << dk[i] << ' ';
    cout << "\n按序遍历顺序一致: " << (dk == mk ? "是" : "否") << endl;
}

```

lab12.cpp(第四段:任务4 计时 + 任务5 删除全部)

```

// 任务 4:10^4 随机键计时对比
const int N = 10000;
vector<int> ks(N);
for (int i = 0; i < N; ++i) ks[i] = i;
shuffle(ks.begin(), ks.end(), rng);
SkipDict s2;
auto t0 = Clock::now();
for (int k : ks) s2.insert(k);
double s1Ins = chrono::duration<double, milli>(Clock::now() - t0).count();
map<int, int> m2;
t0 = Clock::now();
for (int k : ks) m2[k] = k;
double m1Ins = chrono::duration<double, milli>(Clock::now() - t0).count();
long long hits = 0;
t0 = Clock::now();
for (int k : ks) hits += s2.find(k);
double s1Find = chrono::duration<double, milli>(Clock::now() - t0).count();
t0 = Clock::now();
for (int k : ks) hits += m2.count(k);
double m1Find = chrono::duration<double, milli>(Clock::now() - t0).count();
cout << "\n== 任务4:10^4 随机键计时 ==" << endl;
cout << "插入 " << N << " 键:跳表 " << s1Ins << " ms,map " << m1Ins << " ms" << endl;
cout << "查找 " << N << " 键:跳表 " << s1Find << " ms,map " << m1Find << " ms" << endl;
cout << "(命中数 " << hits << ",防止死代码消除)" << endl;

// 任务 5:删除全部元素
for (int k : ks) s2.erase(k);
cout << "\n== 任务5:删除全部后 ==" << endl;
cout << "跳表 size=" << s2.size() << ",为空=" << (s2.size() == 0 ? "是" : "否")
    << ",find(0)=" << s2.find(0) << endl;
return 0;
}

```

示例输出(真实运行,机器不同数值不同,量级关系一致):

```

== 任务3:跳表字典 vs std::map 一致性 ==
20000 次随机操作后:跳表 size=9293,map size=9293
跳表前 10 键: 5 22 38 58 68 78 95 96 101 127
按序遍历顺序一致: 是

== 任务4:10^4 随机键计时 ==
插入 10000 键:跳表 3.1292 ms,map 1.2555 ms
查找 10000 键:跳表 1.188 ms,map 0.6599 ms
(命中数 20000,防止死代码消除)

== 任务5:删除全部后 ==
跳表 size=0,为空=是,find(0)=0

```

设计说明:① update 数组统一"下探路径",插入/删除共享骨架;② 插入前先 find 去重,与 map 键唯一语义对齐;③ 任务3 用同一随机操作序列同时驱动两者,任何一次查找不一致立即退出;④ 同一份键序列计时,两者同为 $O(\log n)$,map 常数更小;删除后回收空高层,size=0 验证删除路径正确。

全书十二章到此结束:第1-2章打下递归与复杂度地基,第3-9章建起线性结构、树、散列、堆、排序、图的工具箱,第10-11章学会设计范式与摊还分析,本章补上最后一块拼图——平衡与概率。数据结构是算法工程师的母语,愿你在竞赛、系统设计或工程实践中带着这十二章的判断力走得更远。